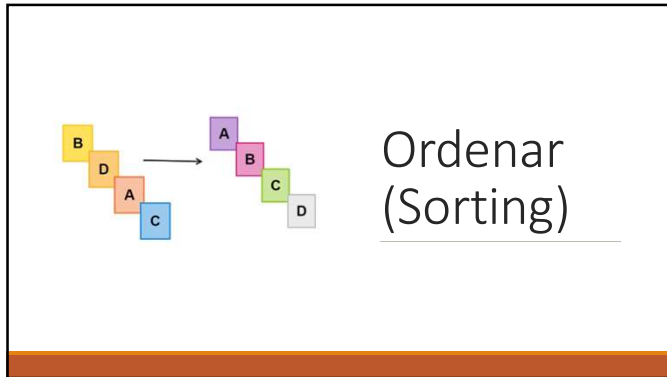
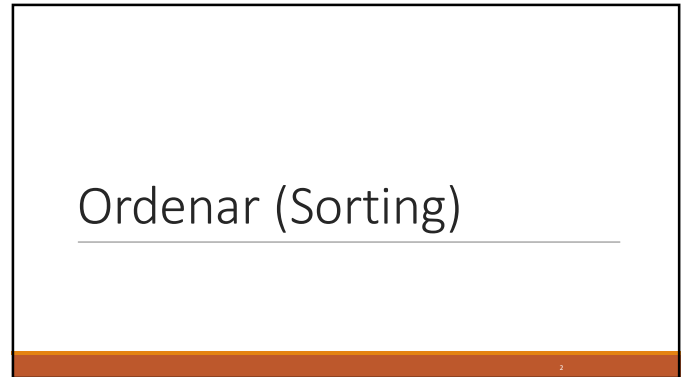




1



2

Ordenar (Sorting)

- Proceso de reorganizar los datos para que estén en orden creciente (o decreciente).
- Ordenar es necesario con frecuencia y es una utilidad importante que hay que conocer.
- Los investigadores han dedicado mucho tiempo a desarrollar algoritmos eficientes para ordenar matrices.

3

Ordenar (Sorting)

- Discutiremos algoritmos cuya complejidad temporal oscila entre $O(N \log N)$ y $O(N^2)$.
- Nos centraremos en el ordenamiento de matrices específicamente.
- La mayoría de estos algoritmos dividen la matriz en submatrices.
- En términos de Java, presumimos que los objetos son Comparables.

4

Comparación de objetos

NO ES LO MISMO QUE EQUALS()

5

Comparable

- Es un Java **interface**.
- Cualquier clase que implemente este interface tendrá el método `compareTo()`.

`x.compareTo(y)`

- Compara los objetos `x` e `y`, donde ambos son del mismo tipo.
 - Si `x < y`, devuelve un valor negativo.
 - Si `x == y`, devuelve 0
 - Si `x > y`, devuelve un valor positivo.

6

Comparator

- También es un Java **interface**.
- Cualquier clase que implemente este interface actúa como un comparador externo para otra clase.
- Tiene el método `compare()`:


```
comp.compare(x, y)
```

 - `comp` es una instancia de la clase que implementa **Comparator**.
 - `x` e `y` son de objetos del mismo tipo.
 - Si `x < y`, devuelve un valor negativo.
 - Si `x == y`, devuelve 0
 - Si `x > y`, devuelve un valor positivo.

7

Comparable vs Comparator

- Una clase que es **Comparable** tiene un punto de comparación establecido a través del método `compareTo()`.
- No podemos cambiar el punto de comparación de `compareTo()`.
- Una clase que es un **Comparator** existe con el propósito de comparar un tipo de objeto específico, que puede o no ser **Comparable**.
- El **Comparator** es como el **Iterator**, existe para ayudar con otra clase.

8

Cuándo usar cada uno

Utilice **Comparable** cuando:

1. Hay un orden "predeterminado" o natural que tiene sentido para la clase.
- Ejemplo: ordenar strings alfabéticamente, enteros por valor o empleados por prioridad.

Utilice **Comparator** cuando:

1. Desea que haya varias formas de ordenar la misma clase.
2. No puede o no quiere cambiar la clase, especialmente si es de una librería.
3. Necesita una ordenación más compleja o dinámica, como las comparaciones condicionales.

9

9

Comparable vs Comparator - Resumen

Característica	Comparable	Comparator
Paquete	java.lang	java.util
Método	compareTo()	compare()
¿Dentro de clase?	Sí	No
¿Orden por defecto (default)?	Sí	No
¿Múltiples estilos de ordenación?	No	Sí
¿Modifica la clase original?	Sí	No

10

10

Comparable vs Comparator

- ¿Por qué usar un **Comparator**?
 - Nos permite comparar clases que no están definidas por nosotros.
 - Nos permite comparar una clase por un atributo diferente.
- Ejemplo 1: Supongamos que desea comparar dos objetos **Rectangle** en Java y no es una clase **Comparable**.
 - Para poder compararlos necesitaríamos crear un **Comparator** de tipo **Rectángulo** y establecer qué vamos a utilizar como comparación.
 - Puede ser: área, perímetro, punto de origen, etc.

11

11

Comparable vs Comparator

- Ejemplo 2: Supongamos que tenemos una clase definida por el usuario llamada **Student** y sus atributos incluyen nombre, número de teléfono y correo electrónico.
 - Esta clase es **Comparable** y su método **compareTo()** utiliza el atributo **name** para comparar **Students**.
 - Por lo tanto, la clase **Student** solo se puede comparar por nombre.
 - Para comparar **students** por correo electrónico o cualquier otro atributo necesitaríamos un **Comparator** que establezca esa comparación.

12

12

El método Equals()

- Este método forma parte de la clase Object.
- Cada clase que definimos hereda de Object, por lo tanto,
 - todas tienen un método equals().
- Si queremos ser capaces de comparar correctamente dos instancias de una clase, necesitamos anular este método y establecer el punto de comparación.

equals()

- Devuelve true si es igual
- False de otra manera

13

El método Equals()

- Por ejemplo, si definimos la clase Student, ¿qué determina si 2 instancias representan al mismo estudiante?
- ¿Sería su nombre? ¿Identificación? ¿Correo electrónico?
- Definimos el equals() para establecer eso.
 - **NOTA: El método equals() solo verifica si son el mismo objeto.**
- Esto equivale a comprobar si compare() o compareTo() devuelve 0.

14

¿Por qué nos importa?

- Las comparaciones son importantes, ya que son esenciales para la ordenación.
- No podemos ordenar si no sabemos cómo determinar si un objeto es más pequeño, igual o más grande.
- El método equals() **no** ayuda para esto, pero se menciona ya que determina la igualdad.
- Ordenar objetos comparables es más sencillo ya que el propio objeto tiene el método necesario para comparar.
- Ordenar usando **Comparator** involucra recibir el Comparator para ordenar basado en eso.

15

Cuándo usar cada uno

Use Comparable cuando: hay UN ORDEN natural

Ejemplos:

- enteros
- Nombres alfabéticos
- Fechas

Use Comparator cuando: necesitas MUCHAS reglas de ordenación

Ejemplos:

- Ordena a los estudiantes por:
 - puntuación
 - Nombre
 - Edad
 - ID

16

Métodos de ordenación

17

Métodos Comunes

Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Heap Sort

18

Selection Sort

19

Selection Sort

- Algoritmo relativamente sencillo de entender.
- Pasos:
 - Pasa varias veces a través de la matriz.
 - Busca el valor más pequeño de la matriz.
 - Coloca ese valor en la posición correcta.
- Esto implica dos procesos: búsqueda e intercambio (swapping).

20

Selection Sort

- Dividiremos la matriz en 2 sub-matrices: matriz ordenada y matriz sin ordenar.
- Inicialmente, toda la matriz forma la matriz sin ordenar.



21

Selection Sort

- La idea es que buscaremos en la matriz sin ordenar para encontrar el valor más pequeño allí.



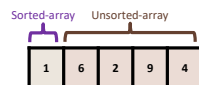
- Una vez encontrado, se intercambia con el valor en la primera posición de la matriz sin ordenar.



22

Selection Sort

- Este valor ahora se une a la submatriz de matriz ordenada.



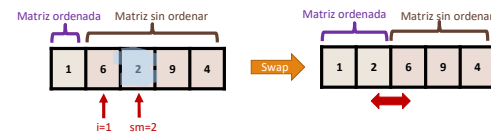
- Repetimos este proceso hasta llegar al final de la matriz.
- un pseudocódigo de este proceso.

```
for i = 0 to n - 2
  # i represents beginning of unsorted portion
  sm ← index of smallest value from A[i] to A[n-1]
  swap A[sm] with A[i]
```

23

Selection Sort

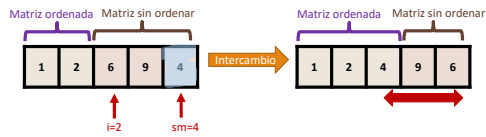
```
for i = 0 to n - 2
  # i represents beginning of unsorted portion
  sm ← index of smallest value from A[i] to A[n-1]
  swap A[sm] with A[i]
```



24

Selection Sort

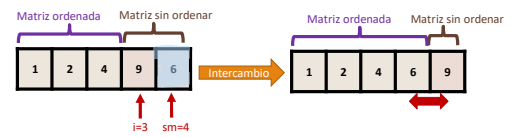
```
for i = 0 to n - 2
  # i represents beginning of unsorted portion
  sm ← index of smallest value from A[i] to A[n-1]
  swap A[sm] with A[i]
```



25

Selection Sort

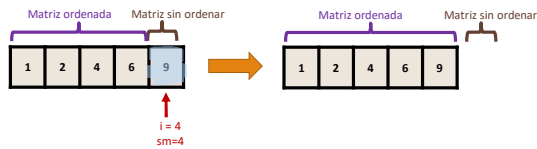
```
for i = 0 to n - 2
  # i represents beginning of unsorted portion
  sm ← index of smallest value from A[i] to A[n-1]
  swap A[sm] with A[i]
```



26

Selection Sort

```
for i = 0 to n - 2
  # i represents beginning of unsorted portion
  sm ← index of smallest value from A[i] to A[n-1]
  swap A[sm] with A[i]
```



27

Selection Sort



- Cuando lleguemos al final de la matriz, se ordenará toda la matriz.
- La complejidad temporal de esto es $O(N^2)$ debido al número de comparaciones que debemos hacer.

28

Insertion Sort

29

Insertion Sort

- Pasos:
- Divida la matriz en ordenada y no ordenada.
- Los ordenados inicialmente consisten en el primer elemento de la matriz.
- A continuación, tomamos continuamente el primer valor de la matriz sin ordenar y encontramos su posición correcta en la matriz ordenada.
- Esto implica dos procesos: búsqueda y desplazamiento.

30

Insertion Sort

- Nuevamente dividimos la matriz en 2 sub-matrices: ordenada y matriz sin ordenar.
- Inicialmente, el primer valor representa la matriz ordenada y el resto la matriz no ordenada.



31

Insertion Sort

- La idea es que tomaremos el primer valor en la matriz sin ordenar y lo desplazaremos a través de la matriz ordenada hasta que esté en la posición correcta.



- A diferencia de selection sort, no intercambiamos solo dos valores, sino que intercambiamos varios valores.

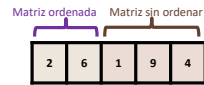


En este caso, el 6 ya estaba en la posición correcta, no fue necesario cambiar.

32

Insertion Sort

- La posición en la que originalmente estaba el primer valor (6) ahora forma parte de la submatriz de matriz ordenada.



- Repetimos este proceso hasta llegar al final de la matriz.
- Aquí hay un pseudocódigo de este proceso.

```

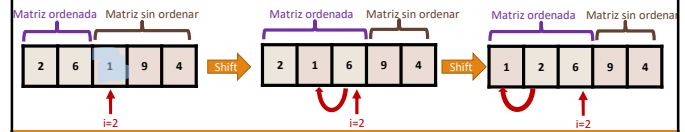
for i = 1 to n - 1
  # i represents beginning of unsorted portion
  valToInsert = A[i]
  j = i - 1
  while j >= 0 and A[j] > valToInsert
    # Keep moving values to the right, making room for valToInsert
    A[j + 1] = A[j]
    j = j - 1
  A[j + 1] = valToInsert
  
```

33

Insertion Sort

```

for i = 1 to n - 1
  # i represents beginning of unsorted portion
  valToInsert = A[i]
  j = i - 1
  while j >= 0 and A[j] > valToInsert
    # Keep moving values to the right, making room for valToInsert
    A[j + 1] = A[j]
    j = j - 1
  A[j + 1] = valToInsert
  
```

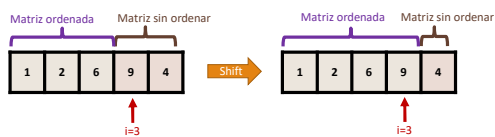


34

Insertion Sort

```

for i = 1 to n - 1
  # i represents beginning of unsorted portion
  valToInsert = A[i]
  j = i - 1
  while j >= 0 and A[j] > valToInsert
    # Keep moving values to the right, making room for valToInsert
    A[j + 1] = A[j]
    j = j - 1
  A[j + 1] = valToInsert
  
```

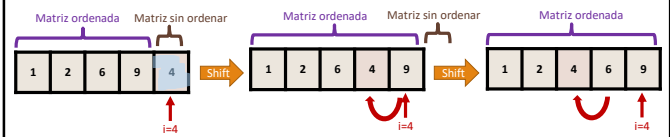


35

Insertion Sort

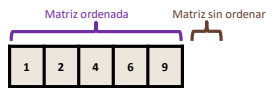
```

for i = 1 to n - 1
  # i represents beginning of unsorted portion
  valToInsert = A[i]
  j = i - 1
  while j >= 0 and A[j] > valToInsert
    # Keep moving values to the right, making room for valToInsert
    A[j + 1] = A[j]
    j = j - 1
  A[j + 1] = valToInsert
  
```



36

Insertion Sort



- Cuando lleguemos al final de la matriz, se ordenará toda la matriz.
- Dio un paso menos.
- La complejidad temporal de esto es $O(N^2)$ debido al número de comparaciones que debemos hacer (desplazamiento).

37

37

Selection vs Insertion

- Ambos algoritmos tienen una complejidad temporal de $O(N^2)$.
- En el caso de matrices pequeñas, ambos funcionan de la misma manera.
- A medida que las matrices se hacen más grandes, se hace más notable que insertion sort es ligeramente mejor que selection.
- Esto se debe a que insertion se beneficia cuando las matrices están parcialmente ordenadas.

38

38

Merge Sort

39

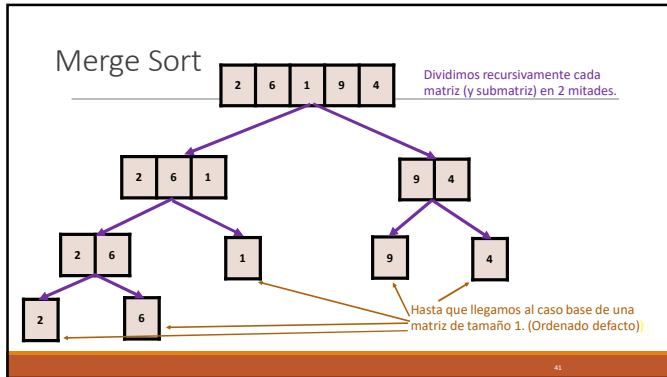
39

Merge Sort

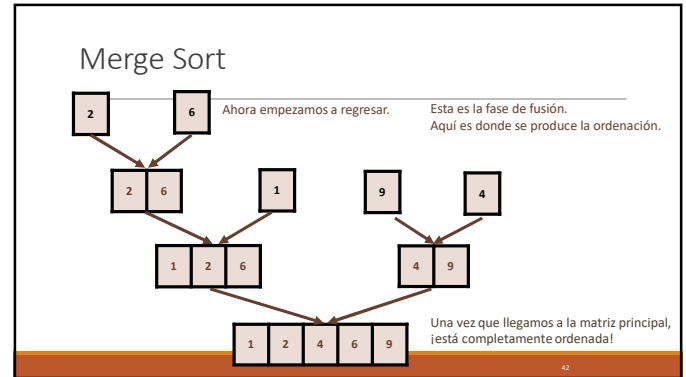
- Ahora cambiamos la estrategia a Divide y Vencerás.
- La idea general aquí es que es más fácil ordenar matrices más pequeñas, así como datos ya parcialmente ordenados.
- Pasos:
 - Particionar de forma recursiva la matriz en dos mitades.
 - Ordene los datos a medida que se vuelven a fusionar.

40

40



41



42

Merge Sort

- Este algoritmo es $O(N \log N)$.
- Es **log N** gracias a dividir la matriz por la mitad cada vez.
- Y **N** para la clasificación en sí.
- Puede ser intenso en la memoria si tuviéramos que crear matrices temporales para las mitades. No es un problema si se hace en el lugar (in-place).

43

Quick Sort

44

Quick Sort

- También divide la matriz en mitades.
- La idea es configurar un valor de pivote y ordenar parcialmente la matriz en función de ese pivote.
- A continuación, "dividimos" la matriz en izquierda y derecha.
- El lado izquierdo contiene valores más pequeños que el pivote, mientras que el lado derecho contiene valores más grandes que el pivote.
- Una vez hecho esto, colocamos el pivote entre estos 2 sub-arrays.
- Luego repetimos el proceso para las sub-matrices a la izquierda y a la derecha del pivote.

45

Quick Sort - Ejemplo

Ordenar esta lista: [5, 2, 9, 1, 7]

1. Elegir pivote → 5
2. Partir:
 - Menos de 5: [2, 1]
 - Mayor que 5: [9, 7]
3. Ordenar [2, 1] → [1, 2]
4. Ordenar [9, 7] → [7, 9]
5. Combinar: [1, 2] + 5 + [7, 9] → **[1, 2, 5, 7, 9]**

46

Quick Sort

47

Quick Sort – En detalle

- Pasos:
- Elija un valor para que sea un pivote, en nuestro caso, la última posición.
- Entonces tenemos 2 índices:
 - Izquierda: Comienza desde la primera posición
 - Derecha: Comienza desde la posición: última - 1
- La izquierda se mueve una posición a la derecha (izquierda + 1) si apunta a un valor menor que el pivote.
- La derecha se mueve una posición a la izquierda (derecha - 1) si apunta a un valor mayor que el pivote.

48

Quick Sort

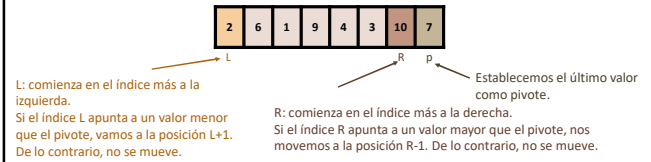
- Pasos:
 - Si la izquierda y la derecha se detienen y aún no se han cruzado, intercambiamos sus valores.
 - Si la izquierda y la derecha se cruzan (izquierda > derecha), entonces cambiamos el pivote por la izquierda.
 - En este punto, se garantiza que la izquierda está en un valor mayor que el pivote.
 - Repita en la submatriz a la izquierda para pivotar y en la submatriz a la derecha para pivotar.

49

51

Quick Sort

Configuración inicial

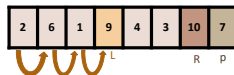


50

50

Quick Sort

Primero vamos a mover el índice L hasta que alcancemos un valor mayor que el pivote.



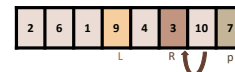
En este caso nos detenemos en la posición 3 donde tenemos el valor 9, que es mayor que el pivote (7).

51

51

Quick Sort

Ahora movemos el índice R hasta que alcancemos un valor menor que el pivote.



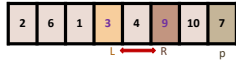
En este caso nos detenemos en la posición 5 donde tenemos el valor 3, que es menor que el pivote (7).

52

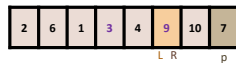
52

Quick Sort

Dado que $L < R$, todavía estamos ordenando, por lo tanto, intercambiamos los valores.



Comenzamos a mover L de nuevo.

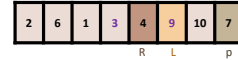


Alcanza la misma posición que R. Se detiene en movimiento desde las $9 > 7$.

53

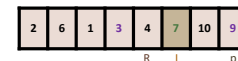
Quick Sort

Comenzamos a mover R de nuevo.



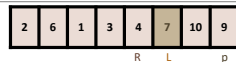
Alcanza la posición 4. Se detiene en movimiento dado que $4 < 7$.

Tanto L como R se han detenido de nuevo. ¡Esta vez han cruzado! ($l > der$). Esto significa que ya tenemos los datos organizados en función del pivote. Podemos cambiar p por L.



54

Quick Sort

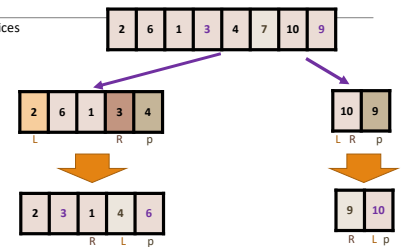


- Fijate en algunas cosas:
- Cuando $L > R$ tenemos garantizadas algunas cosas:
 - Cada valor antes de la posición L es menor que el pivote.
 - Cada valor después de R es mayor que el pivote.
 - R apunta a un valor menor que el pivote y L apunta a un valor mayor que el pivote.
- Teniendo en cuenta esto, podemos intercambiar el valor en L, ya que sabemos que es más grande que el pivote y aún mantenemos el orden general.
- Cuando se intercambia el pivote, ha alcanzado su posición correcta dentro de la matriz.
- Ahora repetimos el proceso para las submatrices a la izquierda y a la derecha del pivote.

55

Quick Sort

¡Repetimos el proceso para las submatrices izquierda y derecha!

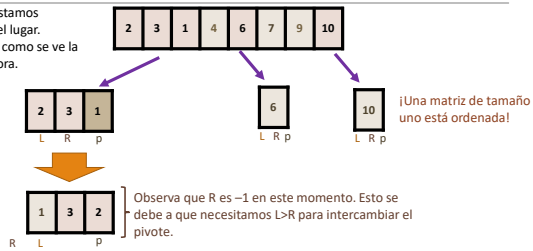


**¡Este es el resultado de esta sub-matriz!

56

Quick Sort

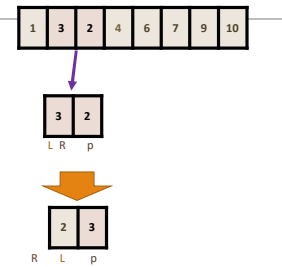
Recuerde que estamos clasificando en el lugar. Entonces, así es como se ve la matriz hasta ahora.



57

Quick Sort

Recuerde que estamos clasificando en el lugar. Entonces, así es como se ve la matriz hasta ahora. Podemos ver que el valor del pivote va de hecho a su posición correcta.



58

Quick Sort



Y finalmente, 3 se ordena de forma predeterminada ¡dándonos la matriz ordenada final!

59

Quick Sort

- Este algoritmo es $O(N^2)$ En el peor de los casos, pero $O(N \log N)$ en promedio.
- $O(N^2)$ se debe a situaciones en las que la mayoría de las veces particionamos solo en 1 sub-matriz. (El pivote va en uno de los extremos de la matriz)
- Es como lo opuesto a merge sort. Ordenamos los datos y luego los dividimos en particiones más pequeñas. Ordenamos desde la matriz más grande hasta la submatriz más pequeña.
- Mientras que la ordenación por fusión primero divide los datos y luego comienza a ordenar desde la submatriz más pequeña hasta la más grande.

60

Quick Sort

- El pivote no tiene por qué ser el último valor, podemos elegir otra posición.
- Algunas posiciones para el pivote son más ideales que otras.
- Cambiar la posición del pivote también afecta con qué valor cambiamos el pivote cuando terminamos de ordenar los datos.
- Merge-sort es más eficiente y rápido para matrices grandes.
- Quicksort es más eficiente y rápido para matrices más pequeñas.

61

61

Merge vs Quick Sort

- Merge particiona los datos en $N/2$ cada vez.
- En Quicksort, las particiones pueden ser de cualquier tamaño.
- Merge puede ocupar memoria adicional si no se hace en el lugar (generar submatrices)
- Quicksort está hecho para hacerlo en su lugar (in-place).

62

62

Heap Sort

63

63

Heap Sort

- Para este algoritmo aprovechamos el Heap ADT.
- Heaps son árboles completos que organizan los datos colocando en niveles superiores el valor más grande o más pequeño.

64

64

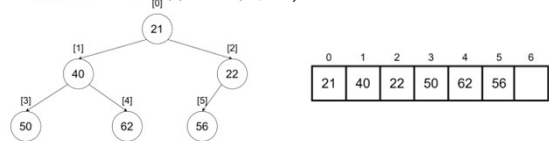
Heap Sort

- Anteriormente demostramos que para un árbol completo, podemos implementarlo fácilmente usando matrices.
- Esto se debe a que los árboles completos se construyen por nivel.
- Teniendo en cuenta esto, encontrar la ubicación de un nodo que se está agregando es tan simple como colocar ese valor en el siguiente espacio vacío disponible.

65

Heap Sort

- En resumen, la posición de cada nodo se puede calcular como:
 - hijo izquierdo: $2i + 1$
 - Hijo Derecho: $2i + 2$
 - Padre: $\text{floor}((i - 1) / 2)$



66

Heap Sort

- ¿Cómo funciona esta ordenación??
- La idea es la siguiente:
 1. Tomamos la matriz sin ordenar y la tratamos como un árbol completo.
 2. heapify el Árbol para convertirlo en unMax Heap.
 3. Ordenamos la matriz eliminando la raíz y luego colocándola al final de la matriz.

67

Heap Sort

Supongamos que se trata de la matriz sin ordenar:

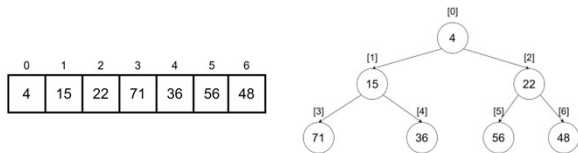
0	1	2	3	4	5	6
4	15	22	71	36	56	48

68

Heap Sort

Paso 0: Tratar la matriz como un árbol completo.

- Nota: No vamos a cambiar nada de la matriz.



69

Heap Sort

Paso 1: heapify el Árbol.

- Este proceso sigue la idea previa de primero garantizamos un Árbol completo.
- Luego lo arreglamos.
- En este caso, la matriz ya es un árbol completo.
- Ahora lo arreglamos, por lo que se convierte en un Heap.

70

Heap Sort

Paso 1: heapify el Árbol.

- heapify el Árbol es aplicar **sifting down** en cada nodo interno.
- El proceso es el siguiente:
 - Comenzamos en el último nodo interno. Esto está en la posición $(n / 2) - 1$. ¿Por qué empezar aquí?
 - El ir desde ese nodo hasta la raíz $(n/2 - 1$ a 0) aplicamos sifting down.

71

Heap Sort

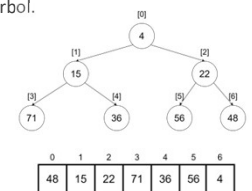
Paso 1: heapify el Árbol.

Ejemplo: Supongamos que tenemos este árbol.

$n = 7$

Último nodo interno: $i = 6/2 - 1 = 2$

Empezaremos heapifying en $i = 2$.

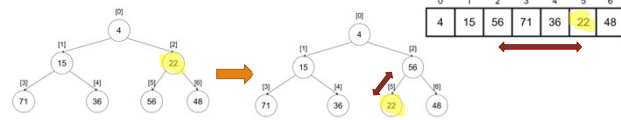


72

Heap Sort

Paso 1: heapify el Árbol.

Ejemplo: Supongamos que tenemos este árbol.
heapify $i = 2$.

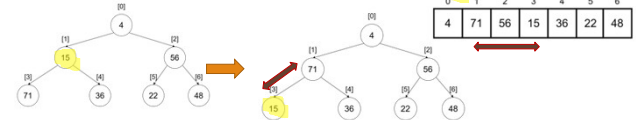


73

Heap Sort

Paso 1: heapify el Árbol.

Ejemplo: Supongamos que tenemos este árbol.
heapify $i = 1$.

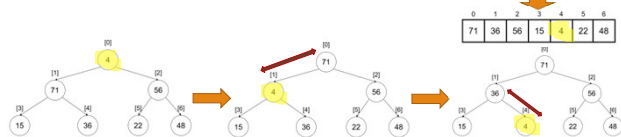


74

Heap Sort

Paso 1: heapify el Árbol.

Ejemplo: Supongamos que tenemos este árbol.
heapify $i = 0$.



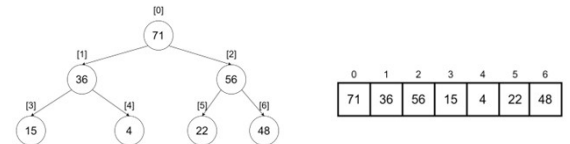
75

Heap Sort

Paso 1: heapify el Árbol.

Ejemplo: Supongamos que tenemos este árbol.

Después de este proceso, el árbol ahora es un Heap máximo.

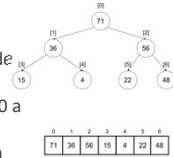


76

Heap Sort

Paso 2: Ordenar la matriz.

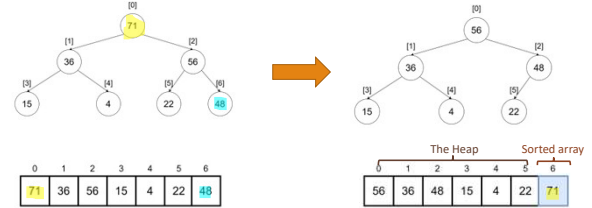
- "Eliminamos" del heap.
- Esto significa quitar la raíz y arreglar el heap.
- Ese nodo "eliminado" se convierte en el último valor de la matriz.
- En términos del heap, Ahora consta de las posiciones 0 a n-1.
- Observe que el nodo se "elimina" del árbol, pero no la matriz.



77

Heap Sort

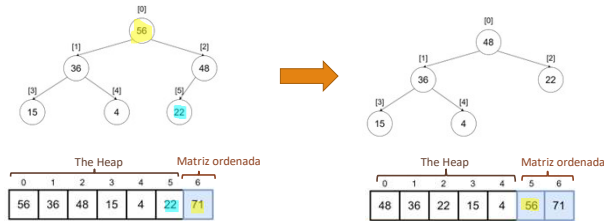
Paso 2: Ordena la matriz.



78

Heap Sort

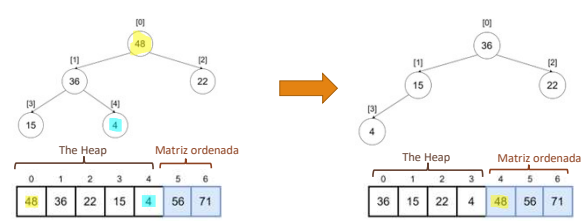
Paso 2: Ordena la matriz.



79

Heap Sort

Paso 2: Ordena la matriz.

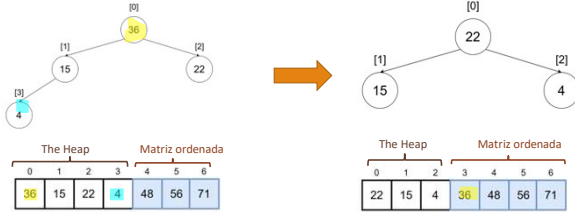


80

80

Heap Sort

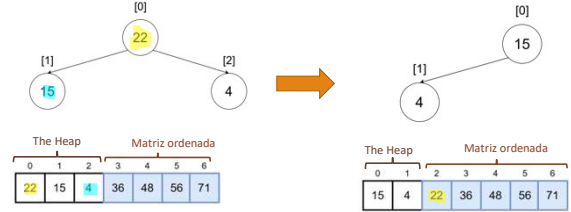
Paso 2: Ordena la matriz.



81

Heap Sort

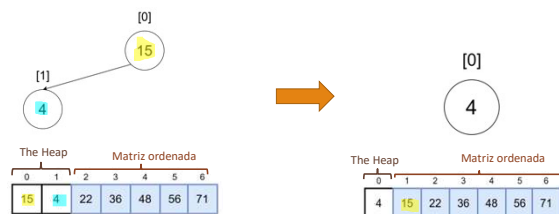
Paso 2: Ordena la matriz.



82

Heap Sort

Paso 2: Ordena la matriz.



83

Heap Sort

Paso 2: Ordena la matriz.

0	1	2	3	4	5	6
4	15	22	36	48	56	71

La matriz ahora está ordenada. Tenga en cuenta que no es necesario "eliminar" el último nodo.

84

Heap Sort

- Este algoritmo es $O(N \log N)$. **Por qué?**
 - Cuando ordenamos la matriz, eliminamos la raíz n veces y cuando el árbol se arregla a sí mismo, es $O(\log n)$.
- Al igual que Quicksort, este algoritmo es en su lugar (in-place).

85

Métodos de Ordenación en Java

86

Métodos de Ordenación en Java

- Java ya tiene varios métodos de ordenación incorporados.
- La clase `Array` varios métodos `sort()` sobrecargados.
- Puede ordenar matrices de tipos primitivos (por ejemplo, `int[]`, `float[]`, etc.)
- Si se pasa una matriz de un Objeto, el objeto debe ser `Comparable` o se debe proporcionar un `Comparator`

87

Para primitivos

Java utiliza Dual-Pivot QuickSort.

- Es una variación que utiliza dos pivotes en lugar de uno, lo que a menudo funciona mejor en la práctica.

- **Complejidad del tiempo:**

- Promedio: $O(n \log n)$
- El peor de los casos: $O(n^2)$, pero raro debido a las optimizaciones.

88

Para objetos

Java utiliza **TimSort**.

- TimSort es un algoritmo híbrido (basado en Merge Sort e Insertion Sort).
- Es estable, lo que significa que conserva el orden de los elementos iguales.
- Funciona bien con datos parcialmente ordenados o pequeños.
- **Complejidad del tiempo:**
 - El peor de los casos: $O(n \log n)$
 - En el mejor de los casos (casi ordenado): $O(n)$

89

Métodos de Ordenación en Java

- La clase Collection también tiene un método `sort()`.
- Solo se puede llamar en Listas.
- Espera que el tipo de lista sea **Comparable**, de lo contrario, se debe proporcionar un **Comparator**.
- Usa Timsort.

90

Conclusión

91

Conclusiones

- Es un tema importante ya que la clasificación de datos es una tarea común en computación.
- También es beneficioso saber qué algoritmos son más eficientes en función de la cantidad de datos que estamos procesando.

92